

```
Enqueue (node * front, node * Rear, int x)
```

```
{  
    node * temp;  
    temp = (node*) malloc (sizeof(node));  
  
    if ( temp == NULL )  
    {  
        printf("overflow");  
        exit(1);  
    }  
  
    temp -> data = x;  
    temp -> next = NULL;  
  
    if ( Rear == NULL )  
    {  
        Front = temp;  
        Rear = temp;  
    }  
    else  
    {  
        Rear -> next = temp;  
        Rear = temp;  
    }  
}
```

```
Dequeue (node * Front, node * Rear)
{
    node * temp;

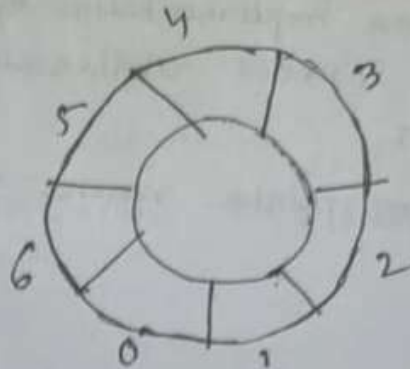
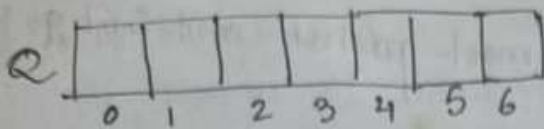
    if ( Front == NULL )
    {
        printf("Underflow");
        exit(1);
    }
    else
    {
        temp = Front;
        Front = Front -> next;

        if ( Front == NULL )    // queue became empty
            Rear = NULL;

        free(temp);
    }
}
```

Circular Queue:

int Q[N], N=7



Overflow Condition.

$$(R+1) \bmod N = F$$

Enqueue:

1. First check for overflow condition. with Mod operation.
2. If it the 1st insertion increment both front and rear. otherwise increment only Rear pointer by using mod and insert the element into queue.

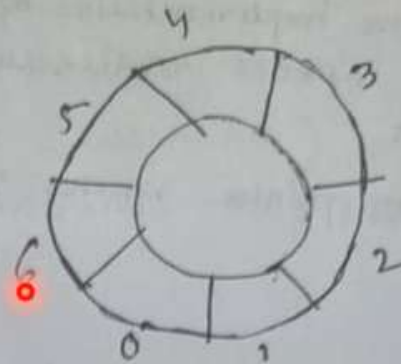
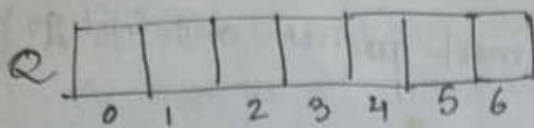
```
Void Circular Enqueue(Q,N,R,F,x)
{
    if ( (R+1) mod N == F )
    {
        pf("over flow");
    }
    else if ( F == -1 && R == -1 )
    {
        F++;
        R++;
        Q[R] = x;
    }
    else
    {
        R = (R+1) mod N;
        Q[R] = x;
    }
}
```

```
int Dequeue(Q, N, R, F)
{
    if (F == -1 && R == -1)
    {
        Printf("underflow");
    }
    else if (F == R)
    {
        x = Q[F];
        F = R = -1;
    }
    else
    {
        x = Q[F];
        F = (F+1) mod N;
    }

    return x;
}
```

Circular Queue:

Int Q[N], N=7



Overflow Condition.

$$(R+1) \bmod N \neq F$$

Enqueue:

1. First check for overflow condition. with Mod operation.
2. if it the 1st insertion increment both front and rear.
otherwise increment only Rear pointer by using mod
and insert the element into queue.

priority queue

- is a type of queue that arranges elements based on their priority values.
- Each element has a priority associated. When we add an item, it is inserted in a position based on its priority.
- Elements with higher priority are typically retrieved or removed before elements with lower priority.
- Binary heap is the most common method to implement a priority queue. In binary heaps, we have easy access to the min (in min heap) or max (in max heap) and binary heap being a complete binary tree are easily implemented using arrays. Since we use arrays, we have cache friendliness advantage also.
- Priority Queue is used in algorithms such as Dijkstra's algorithm, Prim's algorithm, and Huffman Coding.

Implementation of Priority Queue

Input: 1, 5, 3, 2, 6 (Higher number has higher priority)

priority_queue(pq):

enqueue(1)

pq:

1	
---	--

enqueue(5)

pq:

5	1	
---	---	--

enqueue(3)

pq:

5	3	1	
---	---	---	--

enqueue(2)

pq:

5	3	2	1	
---	---	---	---	--

enqueue(6)

pq:

6	5	3	2	1
---	---	---	---	---

Enqueue operation

```
#define N 10

struct Node {
    int value;
    int priority;
};

struct Node Q[N];
```

- `int i = R;` start from the last inserted element
- `while (i >= F && Q[i].priority > pr) →` keep shifting elements one step right until we find the correct position for the new element.

```
void Enqueue(int x, int pr) {
    if (R == N - 1) {
        printf("Overflow\n");
    }
    else if (F == -1 && R == -1) {
        F = R = 0;
        Q[R].value = x;
        Q[R].priority = pr;
    }
    else {
        int i = R;
        while (i >= F && Q[i].priority > pr) {
            Q[i + 1] = Q[i];
            i--;
        }
        Q[i + 1].value = x;
        Q[i + 1].priority = pr;
        R++;
    }
}
```

Insert (10, priority 2)

Queue is empty → goes to step 2.

$Q[0] = \{10, 2\}$
 $F=0, R=0$

Insert (5, priority 1)

Start $i=0$.

Insert (5, priority 1)

Start $i=0$.

Check $Q[0].priority > 1 \rightarrow 2 > 1$ ✓ → shift $Q[0]$ to $Q[1]$.

Now insert (5,1) at $Q[0]$.

$Q[0] = \{5, 1\}$
 $Q[1] = \{10, 2\}$
 $F=0, R=1$

Insert (20, priority 3)

Start $i=1$.

Check $Q[1].priority > 3 \rightarrow 2 > 3$ ✗ → stop.

So insert (20,3) at $Q[2]$.

$Q[0] = \{5, 1\}$
 $Q[1] = \{10, 2\}$
 $Q[2] = \{20, 3\}$
 $F=0, R=2$

Insert (15, priority 2)

Start $i=2$.

- $Q[2].priority > 2 \rightarrow 3 > 2$ ✓ → shift {20,3} to $Q[3]$.
- Now $i=1$. Check $Q[1].priority > 2 \rightarrow 2 > 2$ ✗.
So insert (15,2) at $Q[2]$.

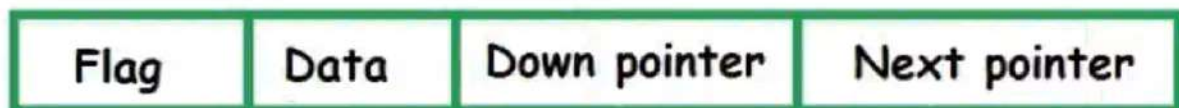
$Q[0] = \{5, 1\}$
 $Q[1] = \{10, 2\}$
 $Q[2] = \{15, 2\}$
 $Q[3] = \{20, 3\}$
 $F=0, R=3$

Dequeue Priority Queue

```
int Dequeue() {  
    if (F == -1 && R == -1) {  
        printf("Underflow\n");  
        return -1;  
    }  
    else {  
        int x = Q[F];  
        if (F == R) {  
            F = R = -1; // queue becomes empty  
        }  
        else {  
            F++;  
        }  
        return x;  
    }  
}
```

Generalized Linked List

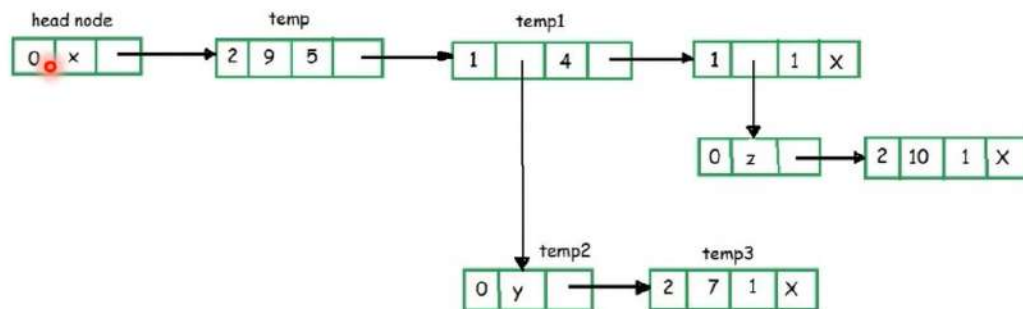
A Generalized Linked List L , is defined as a finite sequence of $n \geq 0$ elements, $l_1, l_2, l_3, l_4, \dots, l_n$, such that l_i are either item or the list of items. Thus $L = (l_1, l_2, l_3, l_4, \dots, l_n)$ where n is total number of nodes in the list.



To represent a list of items there are certain assumptions about the node structure.

- Flag = 1 implies that *down pointer* exists
- Flag = 0 implies that *next pointer* exists
- Flag-2 meaning this is a **coefficient-exponent pair**.
- Data means the item.
- Down pointer is the address of node which is down of the current node
- Next pointer is the address of node which is attached as the next node

Example: $9x^5 + 7x^4y + 10xz$



In the above example the head node is of variable x. The temp node shows the first field as 2 means coefficient and exponent are present.

